프롤로그

서버가 멈추다

밤 열한 시, 막 잠이 들려던 참에 휴대폰이 울렸습니다. 운영팀이었습니다.

운영팀 : "사이트가 전부 멈췄습니다. 확인 좀 해 주세요."

잠이 확 달아났습니다. 노트북을 열자 대시보드가 온통 에러 로그로 덮여 있었습니다. 저녁 일곱 시에 시작한 할인 이벤트로 트래픽이 몰리면서, 그 부하를 버티지 못한 서버가 다운된 것입니다.

서둘러 서버를 재시작했지만 바로 안정되지 않았고, 트래픽이 빠지기 시작한 새벽 한 시가 되어서야 겨우 안정을 되찾았습니다.

트래픽 때문에 사이트 전체가 멈춰 버리다니.

다음 날 아침, 팀장이 자리로 찾아왔습니다.

팀장 : "이번처럼 트래픽 때문에 사이트 전체가 다운되면 곤란해요. 재발 방지 대책 좀 찾아봐 줘요."

선뜻 답할 수 없었습니다. 지난번에도 트래픽이 몰렸을 때 서버를 늘려 봤지만, 비용과 노력만 들었을 뿐 같은 장애가 되풀이됐기 때문입니다. 결국 선배를 찾아갔습니다.

오픈이 : "선배님, 트래픽은 주문에만 몰렸는데 로그인이고 상품 조회고 할 것 없이 전부 멈춰 버렸어요. 이건 어떻게 해야 할까요?"

선배가 웃으며 대답했습니다.

선배 : "모든 기능이 한 서버에 다 뭉쳐 있어서 그래요. 백화점은 정전 한 번에 전 층이 같이 문을 닫지만, 길가의 독립 상점은 한 곳이 닫아도 옆 가게는 멀쩡하잖아요. **기능을 그 상점들처럼 독립된 서비스로 분리하면, 한 곳에 트래픽이 몰려 서버가 다운돼도 다른 기능은 멀쩡하거든요.** 이런 구조를 **MSA**라고 해요. 일단 나누는 것부터 시작해 봐요."

서비스를 나누다

오픈이 : "선배님, 그럼 어떤 것부터 시작하면 될까요?"

선배 : "우선 회원, 주문, 상품, 배달을 기능별로 서비스로 나눠요. 그리고 서비스끼리 전화를 거는 것처럼 직접 호출하게 만드는 거예요."

선배의 말대로 한 서버에 모여 있던 기능을 네 개의 서비스로 나눴습니다. 주문이 들어오면 주문 서비스가 상품 서비스에 재고를 줄여 달라고 요청하고, 이어서 배달 서비스에 배달 생성을 요청하도록 연결했습니다.

그런데 곧 문제에 부딪혔습니다. 중간에 에러가 나면 되돌릴 방법이 없었습니다. 예전처럼 데이터베이스가 하나일 때는 에러가 나면 전부 롤백하면 그만이었지만, 이제는 서비스마다 데이터베이스가 따로 나뉘어 있어 그럴 수가 없었습니다.

오픈이 : "선배님, 재고는 이미 줄였는데 배달이 실패하면 줄인 재고를 되돌릴 방법이 없어요. 어떻게 하죠?"

선배 : "자동으로 안 되니 직접 되돌려야죠. 배달에서 실패하면, 이전 단계인 상품 서비스에 '재고를 복구해 줘'라고 요청을 보내는 거예요. **한 단계씩 거꾸로 취소하는 겁니다.** 이걸 **보상 트랜잭션**이라고 불러요."

주문이 어디까지 진행됐는지 상태를 기록해 두고, 실패하면 이전 서비스로 취소 요청을 보내는 로직을 짰습니다. 단계마다 취소 코드를 일일이 붙이는 건 번거로웠습니다. 그래도 일부러 중간 단계를 실패시켜 보니, 앞서 끝난 단계가 한 단계씩 원래대로 되돌아갔습니다.

그렇게 기능별로 나눈 서비스들이 처음으로 하나의 흐름으로 함께 동작하기 시작했습니다.

구조를 다시 세우다

며칠 뒤, 팀장이 자리로 찾아왔습니다.

팀장 : "주문 기능을 테스트하려는데, 컨트롤러가 서비스에 직접 의존해서 가짜(Mock) 객체로 테스트할 수가 없어요. 이거 직접 의존하지 않게 구조 정리해 줘요."

곧바로 코드를 열어 봤습니다. 컨트롤러를 테스트하려면 서비스까지 함께 동작해야 했고, 가짜 객체를 넣으려 해도 컨트롤러 코드를 직접 고쳐야 했습니다. 고민에 빠진 채 다시 선배를 찾아가 코드를 보여 주며 물었습니다.

선배 : "지금은 결합도가 너무 높아서 그래요. 두 가지만 알면 돼요. 먼저 **구현이 아니라 인터페이스에 의존하게 만들어요.** 그래야 코드를 고치지 않고도 진짜 객체를 가짜 객체로 바꿔 테스트할 수 있어요. 이게 **클린 아키텍처**예요. 그리고 **흩어져 있는 핵심 비즈니스 로직은 도메인 안에 모아 둬요.** 그래야 바깥이 어떻게 바뀌어도 비즈니스 규칙은 흔들리지 않죠. 이걸 **도메인 주도 개발**이라고 해요."

클린 아키텍처와 도메인 주도 개발은 이름만 들어 봤지, 직접 적용해 보긴 처음이었습니다. 두 개념을 이해하고 나서, 익숙한 구조를 하나씩 뜯어고쳤습니다.

컨트롤러가 서비스에 직접 의존하는 대신, 그 사이에 USB 허브 같은 인터페이스를 두었습니다. 허브 뒤의 장치가 바뀌어도 컴퓨터는 아무 영향이 없듯, 컨트롤러는 '무엇을 할지'만 약속한 그 인터페이스에만 의존하고, '어떻게 할지'는 인터페이스를 구현한 서비스 쪽에 맡겼습니다.

검증 규칙도 정리했습니다. 재고 확인이나 주문 검증처럼 서비스 코드 곳곳에 흩어져 있던 규칙을 각 서비스의 도메인 객체 안으로 모았습니다. 규칙이 도메인 안에 모이니, 나중에 규칙이 바뀌어도 도메인만 고치면 됐습니다.

그러자 컨트롤러는 더 이상 서비스에 직접 의존하지 않게 됐습니다. 컨트롤러는 한 줄도 건드리지 않고, 인터페이스를 구현한 서비스만 가짜 객체로 바꿔 끼울 수 있었습니다.

비동기로 전환하다

동기 호출 방식으로 며칠을 운영하던 중, 또 다른 난관이 찾아왔습니다. 상품 서비스가 잠깐 죽은 사이에 들어온 주문이 전부 실패해 버렸습니다. 주문 서비스가 상품 서비스를 직접 호출하고 그 응답을 기다리는 구조라, 상대가 죽어 있으면 호출한 주문도 그대로 실패하는 것입니다.

오픈이 : "선배님, 상품 서비스 하나가 잠깐 죽었을 뿐인데 거기를 호출한 주문까지 다 같이 실패해 버리네요."

선배 : "직접 호출하지 말고, 메시지를 주고받는 비동기 방식으로 바꿔 봐요. **받는 쪽이 잠깐 죽어 있어도 메시지는 그대로 남아 있다가, 서버가 복구되면 그때 처리되거든요. Kafka**를 한번 써 봐요."

Kafka는 처음 다뤄 보는 도구였습니다. 메시지를 발행하고 구독하는 방식을 익힌 뒤, 서비스끼리 직접 호출하던 것을 하나씩 걷어내고 Kafka 메시지로 바꿨습니다.

오픈이 : "그런데 이렇게 메시지로 주고받으면, 중간에 실패했을 때 보상 트랜잭션은 누가 관리하나요?"

선배 : "전체 흐름을 관리하는 지휘자를 하나 두면 돼요. 각 서비스는 자기 일만 하고 결과만 보고하고, 그 지휘자가 어디까지 됐는지에 따라 다음 단계를 진행시키거나, 실패하면 이미 끝난 단계만 되돌리는 거죠."

선배 말대로 전체 흐름을 관리할 **오케스트레이터** 서비스를 만들었습니다. 각 서비스는 자기 일을 끝내면 결과만 메시지로 보고했고, 오케스트레이터는 그 보고를 받아 다음 단계를 진행시키거나, 실패하면 이미 끝난 단계를 되돌렸습니다.

이제 상품 서비스가 잠깐 멈춰도 주문은 메시지로 남아 있다가, 복구되면 하나씩 처리됐습니다. 흐름 중간이 실패해도 오케스트레이터가 끝난 단계만 되돌렸습니다.

실시간으로 알리다

며칠 뒤, 베타 테스터로 새 시스템을 써 본 동료가 떨떠름한 얼굴로 찾아왔습니다.

동료 : "어제 물건을 주문했는데, 화면이 계속 '처리 중'이더라고요. 끝났는지 알 수가 없어서 한참 뒤에 주문 내역을 다시 열어 보고서야 완료된 걸 알았어요."

확인해 보니 서버들끼리는 Kafka로 메시지를 주고받으며 처리를 끝냈지만, 정작 그 사실을 사용자에게는 알리지 않고 있었습니다. 사용자는 처음 받은 '처리 중' 상태에 그대로 멈춰 있었고, 결과를 보려면 직접 주문 내역을 다시 열어야 했습니다.

오픈이 : "선배님, 서버에서는 주문 처리가 완료됐는데 사용자는 처음 '처리 중' 응답만 받아서 끝난 걸 알 수가 없어요. 이건 어떻게 해결하죠?"

선배 : "처리가 끝난 순간 사용자에게 바로 알려 줘야 해요. 서버가 주문 완료 시점을 감지해서, 사용자 화면으로 먼저 말을 걸 수 있게 **WebSocket**을 붙여 봐요."

선배의 말을 듣고 코드를 다시 보니 어긋난 곳이 두 군데였습니다. 주문은 배달이 만들어지는 순간 곧바로 완료로 처리됐고, 정작 그 완료를 사용자에게는 알리지 않았습니다.

먼저 배달이 실제로 끝나는 순간에 주문을 완료 처리하도록 바꿨습니다. 남은 건 그걸 사용자에게 알리는 일이었습니다. WebSocket을 들여다보니 늘 쓰던 HTTP와는 정반대였습니다. HTTP가 한 번 주고받으면 끊기는 편지라면, **WebSocket은 한 번 연결하면 계속 이어지는 전화였습니다.** 연결이 살아 있으니 서버는 변화가 생기는 순간 바로 알릴 수 있었습니다. 그래서 주문이 완료되면 서버가 WebSocket으로 사용자 화면에 알림을 보내도록 연결했습니다.

수정된 코드로 직접 주문을 넣어 보았습니다. 새로고침을 누르지 않았는데도, '처리 중'이던 화면이 '주문 완료'로 바뀌었습니다.

동료 : "이제 새로고침 없이도 주문이 끝난 걸 바로 알 수 있겠네요."

완성된 시스템을 선배에게 보여 주었습니다.

오픈이 : "처음 서버가 멈췄던 밤엔 정말 막막했는데, 한 단계씩 부딪히며 오다 보니 결국 해내게 되네요."

선배는 흐뭇한 표정으로 답했습니다.

선배 : "처음부터 모든 정답을 알고 시작하는 사람은 없어요. 부딪히고, 고치고, 다시 만들다 보면 되는 거예요."